

COMP3211

04. Requirements Engineering

Dr. Max Yu Pei

Intended Learning Outcomes

- ❖ Upon completion of this lecture, you will
 1. understand the concepts of user and system requirements and why these requirements should be written in different ways;
 2. understand the differences between functional and non-functional software requirements;
 3. understand the main requirements engineering activities of elicitation, analysis, and validation, and the relationships between these activities;
 4. understand why requirements change is inevitable and how a requirements traceability matrix can be used to trace requirements changes

Requirements and Requirements Engineering

- ❖ The requirements are the descriptions of the *services* that a system should provide and the *constraints* on its operation. They are essential
 - To ensure everyone shares the same understanding
 - To tell the team what to build and under what limits
 - To make success measurable and testable
 - To reduce failure, rework, and legal exposure
 - To provide the basis for managing the project
- ❖ Requirements engineering (RE): The process of discovering, analyzing, documenting, and checking these services and constraints.

User vs. System Requirements

- ❖ Different kinds of requirements are needed to communicate information about a system to different types of stakeholders.

Aspect	User Requirements	System Requirements
Audience	Customers, users, product owners, business stakeholders.	Developers, architects, testers, requirements engineers, project managers.
Definition	High-level statements of what users need the system to do and any expected constraints.	Detailed, precise descriptions of the services the system must provide and the constraints under which it must operate.
Focus	What the user wants to accomplish.	What the system must do and how well it must do it.
Primary purpose	Communicate needs, define broad scope, gain agreement.	Guide design, implementation, testing, and acceptance.
Language	Natural language, user stories, scenarios, simple diagrams.	Structured natural language, formal models, tables, technical specifications.
Level of detail	Abstract, general, often informal.	Detailed, specific, often structured and technical.
Verifiability	Often difficult to test directly; may be ambiguous.	Should be precise, measurable, and testable.
Relationship	Express the problem or need.	Refine and elaborate user requirements into a buildable specification.

Examples: User and System Requirements

❖ Example 1

- User requirement: A customer should be able to search for products so they can find what they need quickly.
- System requirement: The system shall allow searching by product name, category, and price range. Results shall be returned within 2 seconds for up to 10,000 products, and partial matches shall be supported.

❖ Example 2

- User requirement: A user should be able to mark a task as complete in a to-do app.
- System requirement: The system shall allow a user to select a task and change its status to 'complete.' The task shall then display a checkmark and move to the 'Completed' list. The change shall be saved immediately.

Agile Methods and Requirements

- ❖ Requirements in agile processes
 - Producing detailed system requirements is a waste of time
 - Use incremental requirements engineering and often express requirements as “user stories”
 - Practical for business systems but problematic for many others
- ❖ This lecture presents a “traditional” view of requirements.
 - For most large systems, there is a clearly identifiable requirements engineering phase before system implementation begins
 - The outcome is a requirements document

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional and Non-Functional Requirements

- ❖ Functional requirements are statements of services the system should provide.
 - May state what the system should not do.
- ❖ Non-functional requirements are constraints on the system's services or functions.
 - Often apply to the system, rather than individual features or services.
- ❖ The line between them is not always clear since requirements are not independent
- ❖ The system requirements should include both types of requirements

Functional Requirements

- ❖ Statements of services the system should provide.
 - May be in different forms and at different levels of detail
 - *Functional user requirements* may be high-level statements of what the system should do.
 - *Functional system requirements* should describe the system services in detail.
- ❖ Example
 - Functional user requirement: A customer should be able to cancel an order before it is shipped.
 - Functional system requirement:

FR1: The system shall allow a customer to cancel an order if its status is “Pending” or “Processing” and it has not yet been shipped.

FR2: When an order is cancelled, the system shall change its status to “Cancelled.”

FR3: When an order is cancelled, the system shall release any reserved stock.

FR4: When an order is cancelled and payment has already been taken, the system shall initiate a refund.

FR5: When an order is cancelled, the system shall send a confirmation email to the customer.

FR6: If the order has already been shipped, the system shall display a message stating that cancellation is not allowed.

Non-Functional Requirements

- ❖ Constraints on the system's services or functions.
 - May be more critical than functional requirements
 - It is often more difficult to identify which system components implement specific non-functional requirements
 - Non-functional requirements may affect the overall architecture of a system rather than the individual components.
 - A non-functional requirement may generate other related functional and/or non-functional requirements

Verifiable Non-Functional Requirements

- ❖ Non-functional requirements may be difficult to state precisely, and imprecise requirements may be difficult to verify.
- ❖ Verifiable non-functional requirement: A statement using some measure that can be objectively tested.
- ❖ Example: Usability requirement
 - The online store should load quickly so customers don't abandon their carts.
 - The system shall load any product page within 2 seconds for 95% of requests during a peak load of 10,000 concurrent users.

Metrics for Specifying Non-Functional Requirements

NFR Category	What It Covers	Typical Metrics / Example Targets
Performance	Speed, responsiveness, throughput	Response time $\leq 2s$ for 95th percentile; latency < 200 ms; throughput $\geq 1,000$ requests/sec; CPU $< 70\%$
Availability	Uptime and service continuity	99.9% uptime (≈ 8.76 h downtime/year); 99.99% (≈ 52.6 min/year); planned maintenance windows
Reliability	Frequency of failure and consistency	MTBF $\geq 1,000$ hours; failure rate $< 0.1\%$; defect density < 1 per KLOC
Capacity	Maximum volumes the system can handle	Max 50,000 records; 5 TB storage; 1M transactions/day; 10,000 concurrent sessions
Security	Protection from unauthorized access and attacks	TLS 1.3; AES-256 encryption; MFA; critical vulnerabilities patched $\leq 24h$; lockout after 5 failed logins
Usability	Ease of use and learnability	Task completion $\geq 90\%$; time on task ≤ 2 min; error rate $< 5\%$; SUS score ≥ 80
Accessibility	Inclusive use for people with disabilities	WCAG 2.2 AA; contrast ratio $\geq 4.5:1$; keyboard navigable; screen-reader compatible
Testability	Ease of verifying the system	Automated test coverage $\geq 80\%$; test suite runs ≤ 10 min; defect detection rate $\geq 95\%$
Compliance	Legal, regulatory, contractual obligations	PCI DSS, HIPAA, ISO 27001; 100% audit pass; logs retained ≥ 1 year
Efficiency	Resource consumption	Memory $\leq 80\%$; CPU $\leq 70\%$; battery life $\geq 8h$; energy per transaction $\leq X$ joules
Localization	Language and regional adaptation	Support 10 languages; date/time/currency formats; translation coverage $\geq 98\%$
Safety	Preventing harm to people or environment	SIL 2/3; hazard rate $< 10^{-9}$ per hour; fail-safe behavior verified

Challenges in Specifying Non-Functional Req.

- ❖ Customers for a system may find it difficult to translate their goals into measurable requirements.
 - No simple metrics for some goals (e.g., maintainability)
 - Unclear connection between metrics and everyday experience
 - Too high costs for verification

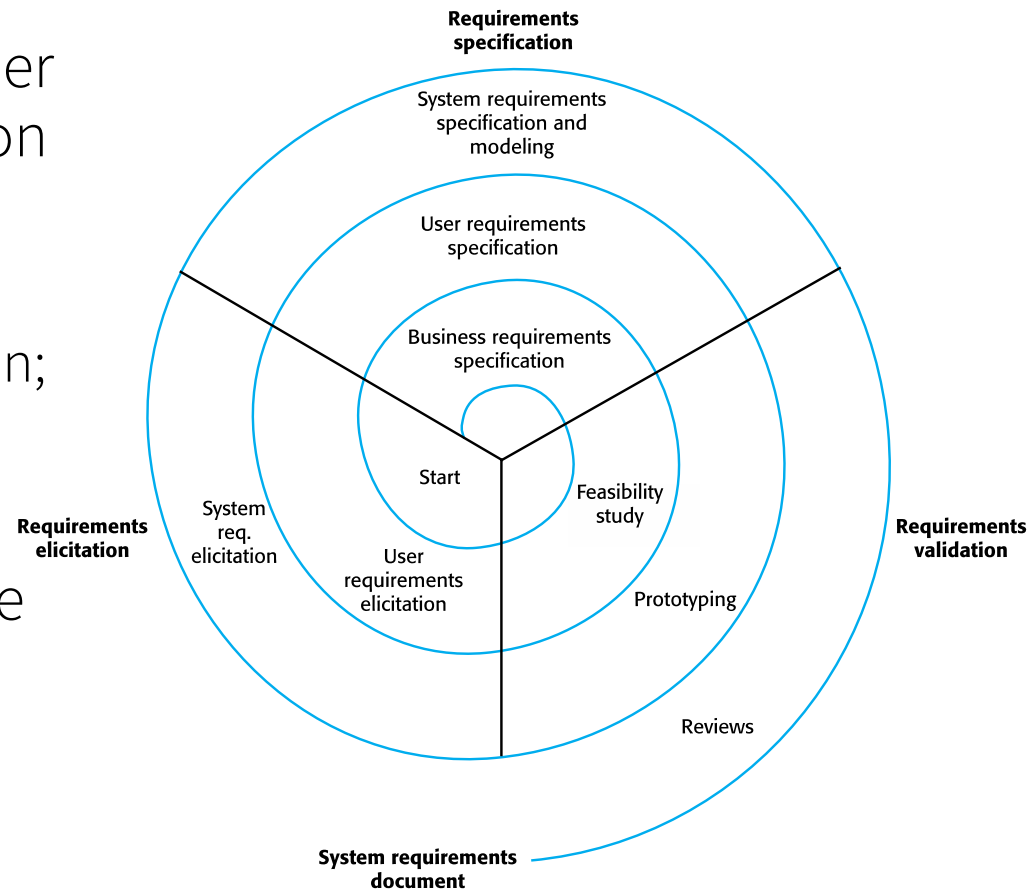
A High-Frequency Trading (HFT) System

- Performance: 99.99% of all trades must be executed within 100 microseconds of receiving the market signal.
 - Availability: The system must have 99.999% ("five nines") uptime during trading hours (9:30 AM - 4:00 PM).
- ❖ It is difficult to separate functional and non-functional requirements in the requirements document.

REQUIREMENTS ENGINEERING PROCESSES

Requirements Engineering Processes

- ❖ The processes used for RE vary widely
- ❖ However, there are a number of generic activities common to all processes
 - Requirements elicitation;
 - Requirements specification;
 - Requirements validation;
- ❖ In practice, RE is an iterative activity in which these processes are interleaved.



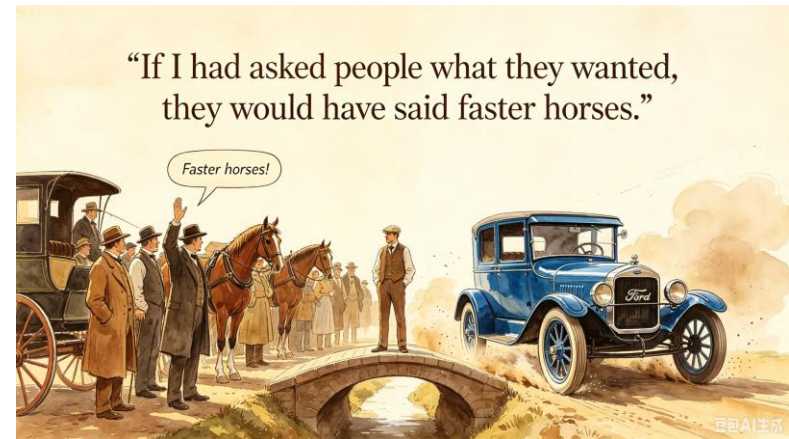
A spiral view of the RE process

REQUIREMENTS ELICITATION

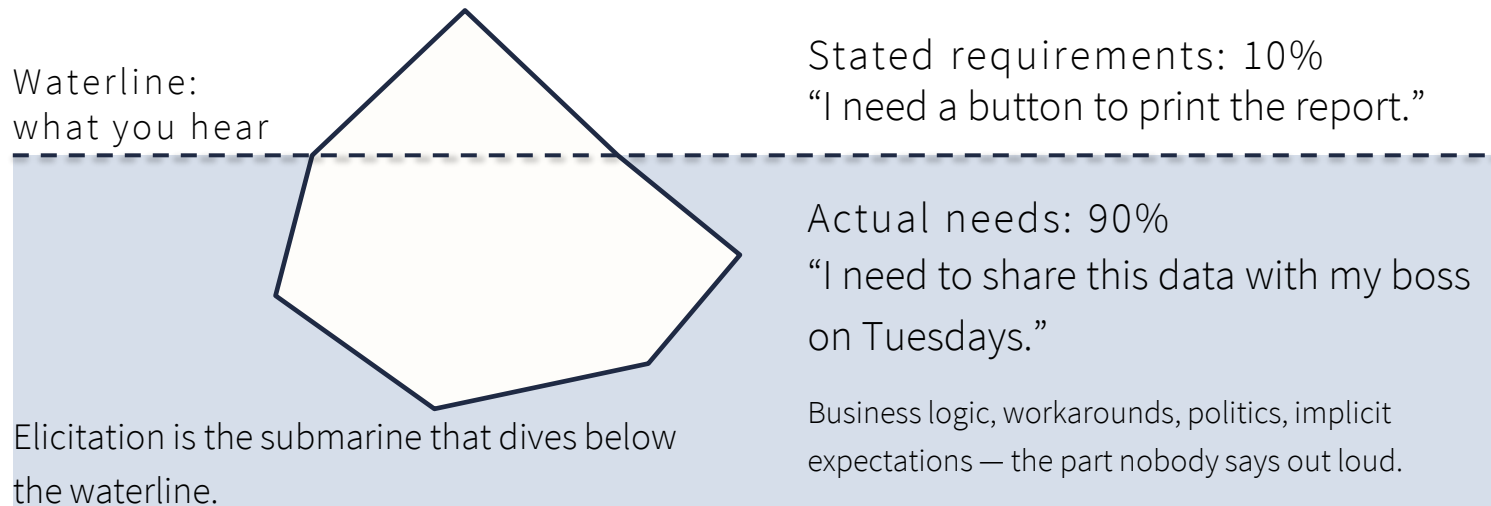
Why Requirements Elicitation Matters

❖ The "Faster Horse" problem

- Users often describe surface-level solutions rather than the root problem.



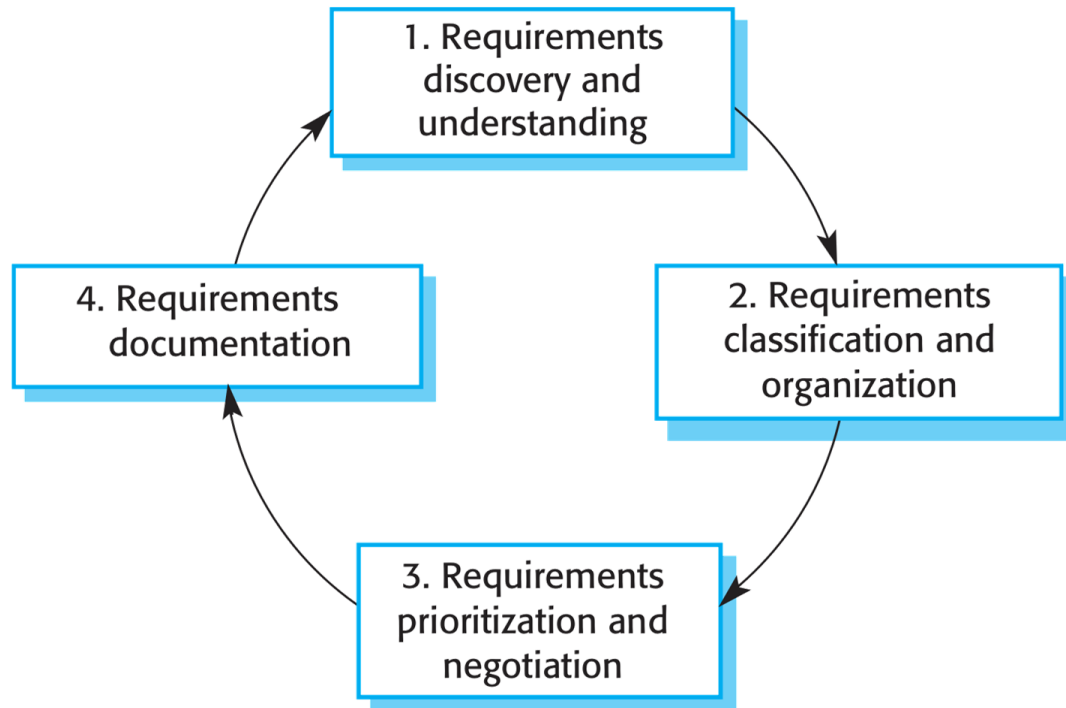
❖ The Requirements iceberg model



Requirements Elicitation

- ❖ Involves technical staff working with stakeholders to understand the application domain, work activities, services, desired system features, and other relevant aspects.

- ❖ Stages



Stakeholder

- ❖ An individual, group, or organization that may affect, be affected by, or perceive itself to be affected by a decision, activity, or outcome of a project
 - Primary users: The people who will actually use the software, every day.
 - Secondary users: Don't use it directly but feed it or depend on it (e.g., suppliers and managers).
 - Domain experts: Know the rules of the business (e.g., pricing, law, workflow, exceptions).
 - Regulatory bodies: Government and legal constraints. Non-negotiable. Often forgotten.
 - Opponents to the project

Core Techniques

- ❖ The analyst's toolbox
 - Interviews: talk to people, one-on-one, in depth
 - Observation: watch real work in its natural setting
 - Surveys: ask many people the same questions
 - Prototyping: show a mock-up, react to something concrete
 - Brainstorming: generate ideas as a group, fast
 - Document analysis: read the manuals and forms already in use

Interviews

- ❖ Talk to people, one-on-one, in depth
 - Open interview: no fixed script or agenda, with open-ended questions
 - Closed interview: predefined set of closed-ended questions
 - Most requirements interviews mix open questions for exploration and closed questions for confirmation
- ❖ Strengths
 - Deep understanding of context and rationale
 - Room for follow-up questions when answers surprise you
 - Builds rapport and trust for the whole project
- ❖ Weaknesses
 - Time-consuming to run, record, and analyze
 - Answers shaped by interviewer and interviewee bias

Observation

- ❖ Ethnography: watch users doing real work, in their natural environment — without interrupting.
- ❖ Strengths
 - People do not have to explain or articulate their work.
 - It is relatively inexpensive.
 - The observer will get a practical insight into the work. Social and organizational factors of importance may be observed.
- ❖ Weaknesses
 - Participants might get disturbed.
 - Participants might change their way of working during observation, and the observer might not get a clear picture.
 - Knowledge-based activities are hard to observe.
 - It is not effective for discovering broader organizational or domain requirements or for suggesting innovations.

Challenges in Requirements Elicitation

- ❖ Stakeholders don't know what they really want.
- ❖ Stakeholders express requirements in their own terms and with implicit knowledge of their work.
- ❖ Different stakeholders with diverse requirements may express their requirements in different ways.
- ❖ Organizational and political factors may influence the system requirements.
- ❖ The requirements change during the analysis process. New stakeholders may emerge, and the business environment may change.

REQUIREMENTS SPECIFICATION

Requirements Specification

- ❖ The process of writing down the user and system requirements in a requirements document.
 - User requirements must be understandable by end-users and customers who do not have a technical background.
 - System requirements are more detailed requirements and may include more technical information.
- ❖ Common notations for writing system requirements
 - Natural language sentences
 - Structured natural language
 - Graphical notations
 - Mathematical specifications

Natural Language Specification (1)

❖ Natural languages are expressive, intuitive, and universal, but ...

Category	Included Limitations	Core Problem
Meaning & Interpretation	Ambiguity; vagueness/imprecision; context-dependence; implicitness; domain jargon	The same words can mean different things to different readers, and important meaning is often left unstated.
Logic & Consistency	Incompleteness; inconsistency; scope/quantifier ambiguity; temporal ambiguity; modality confusion	Requirements may not form a coherent, complete, or logically consistent set.
Specification & Verification	Lack of formal structure; non-verifiability; mixing functional and non-functional requirements	Statements are hard to analyze, test, trace, or automate, and quality constraints become entangled with behaviour.
Human & Organizational	Social/political distortion	People may hide, soften, or avoid real requirements for political or organizational reasons.
Documentation & Maintenance	Verbosity and redundancy	Long, repetitive documents are hard to read, validate, maintain, and change.

Natural Language Specification (2)

- ❖ Guidelines on writing natural language requirements
 - Invent a standard format and use it for all requirements.
 - Use language in a consistent way. Use “shall” for mandatory requirements; Use “should” for desirable requirements.
 - Use text highlighting to identify key parts of the requirement.
 - Avoid the use of computer jargon.
 - Include an explanation (rationale) of why a requirement is necessary.

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. *(Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)*

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. *(A self-test routine can discover hardware and software problems and alert the user to the fact that normal operation may be impossible.)*

Structured Natural Language Specifications

- ❖ An approach to writing requirements where the requirements are written in a standard way rather than as free-form text.
 - Maintains most expressiveness and understandability of NL
 - Removes some of the problems of the NL specification
 - It is still sometimes difficult to write requirements in a clear and unambiguous way

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action:	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered. (see Figure 4.14)
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Precondition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Postcondition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

Use Cases

- ❖ A use case is a structured description of how an external actor interacts with a system to achieve a specific goal

<i>Use case name</i>	ReportEmergency
<i>Participating actors</i>	Initiated by FieldOfficer Communicates with Dispatcher
<i>Flow of events</i>	1. The FieldOfficer activates the “Report Emergency” function of her terminal. 2. FRIEND responds by presenting a form to the FieldOfficer. 3. The FieldOfficer completes the form by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form. 4. FRIEND receives the form and notifies the Dispatcher. 5. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response and acknowledges the report. 6. FRIEND displays the acknowledgment and the selected response to the FieldOfficer.
<i>Entry condition</i>	• The FieldOfficer is logged into FRIEND.
<i>Exit conditions</i>	• The FieldOfficer has received an acknowledgment and the selected response from the Dispatcher, OR • The FieldOfficer has received an explanation indicating why the transaction could not be processed.
<i>Quality requirements</i>	• The FieldOfficer’s report is acknowledged within 30 seconds. • The selected response arrives no later than 30 seconds after it is sent by the Dispatcher.

<i>Use case name</i>	ReportEmergency
<i>Participating actors</i>	Initiated by FieldOfficer Communicates with Dispatcher
<i>Flow of events</i>	1. The FieldOfficer activates the “Report Emergency” function of her terminal. 2. FRIEND responds by presenting a form to the officer. <i>The form includes an emergency type menu (general emergency, fire, transportation) and location, incident description, resource request, and hazardous material fields.</i> 3. The FieldOfficer completes the form by <i>specifying minimally the emergency type and description fields</i>. The FieldOfficer may also describe possible responses to the emergency situation <i>and request specific resources</i>. Once the form is completed, the FieldOfficer submits the form. 4. FRIEND receives the form and notifies the Dispatcher <i>by a pop-up dialog</i>. 5. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. <i>All the information contained in the FieldOfficer’s form is automatically included in the Incident. The Dispatcher selects a response by allocating resources to the Incident (with the AllocateResources use case) and acknowledges the emergency report by sending a short message to the FieldOfficer.</i> 6. FRIEND displays the acknowledgment and the selected response to the FieldOfficer.
<i>Entry condition</i>	• ...

The Software Requirements Specification (SRS)

- ❖ The official statement of what the system developers should implement
 - The information in the SRS depends on the type of system and the development process used.
 - Should include both user and system requirements.
 - Requirements specification standards have been designed (e.g., IEEE Std 830-1998 and ISO/IEC/IEEE 29148).
- It is NOT a design document. As far as possible, it should set out WHAT the system should do rather than HOW it should do it.

The Structure of A Requirements Specification (1)

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

The Structure of A Requirements Specification (2)

Chapter	Description
System requirements specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirements and Design

- ❖ In principle, requirements should state what the system should do and the design should describe how it does this.
- ❖ In practice, requirements and design are inseparable
 - An initial architecture is often needed just to structure the requirements spec.
 - Interoperability with existing systems constrains the design and imposes new requirements.
 - Some non-functional requirements, e.g., reliability via N-version programming, force a specific architecture.
- ❖ A design element in a requirements spec is acceptable if it is explicit, justified, and traceable.
 - It's a defect if it's smuggled in silently.

REQUIREMENTS VALIDATION

Requirements Validation

- ❖ The process of checking that the requirements define the system that the customer really wants
 - Overlaps with elicitation and analysis

- ❖ Different types of checks
 - Validity checks. Do the requirements reflect the real needs of system users?
 - Consistency checks. Are there any requirements conflicts?
 - Completeness checks. Are all functions required by the customer included?
 - Realism checks. Can the requirements be implemented, given the available budget and technology?
 - Verifiability checks. Can the requirements be verified?

Requirements Validation Techniques

- ❖ Various techniques that can be used individually or together
 - Requirements reviews: Systematically and manually analyze the requirements to check for errors and inconsistencies
 - Should be held regularly
 - Should involve the client and contractor staff
 - May be formal (with completed documents) or informal.
 - Prototyping: Using a model of the system to check requirements.
 - Test-case generation: Developing tests for requirements to check verifiability.

- ❖ You rarely find all requirements problems during the requirements validation process!

MANAGE REQUIREMENTS CHANGE

Requirements Traceability Matrix

- ❖ A table that maps requirements to other artifacts in the development lifecycle, including design elements, code, test cases, and other requirements.

Req ID	Requirement	Source	Subsystem	Design element	Code module	Test case	Status
FR-01	...	...		...	...	...	...
...	...	...		...	...	...	...

- Backward traceability: where a requirement came from.
 - Forward traceability: where a requirement goes.
- ❖ Why it matters
 - Static correctness. To assess/demonstrate the system's completeness, correctness, verification, and compliance.
 - Dynamic resilience. To support impact analysis, change propagation, scope control, and progress tracking.

Key Points (1)

- ❖ Requirements for a software system set out what the system should do and define constraints on its operation and implementation.
- ❖ Functional requirements are statements of the services that the system must provide or are descriptions of how some computations must be carried out.
- ❖ Non-functional requirements often constrain the system being developed and the development process being used.
- ❖ Non-functional requirements often relate to the emergent properties of the system and therefore apply to the system as a whole.
- ❖ The requirements engineering process is an iterative process that includes requirements elicitation, specification, and validation.

Key Points (2)

- ❖ Requirements elicitation is an iterative process that can be represented as a spiral of activities – requirements discovery and understanding, requirements classification and organization, requirements prioritization and negotiation, and requirements documentation.
- ❖ You can use various techniques for requirements elicitation, including interviews and ethnography.
- ❖ Requirements specification is the process of formally documenting the user and system requirements and creating a software requirements document.

Key Points (3)

- ❖ The software requirements specification is an agreed statement of the system requirements. It should be organized so that both system customers and software developers can use it.
- ❖ Requirements validation is the process of checking the requirements for validity, consistency, completeness, realism, and verifiability.
- ❖ A requirements traceability matrix is a table that maps requirements to other artifacts in the development lifecycle, including design elements, code, test cases, and other requirements.

THE END